[COLLEGE / INSTITUTE NAME]
[Department of _____]

A Mini Project Report On

Motion Detection and Face Identification using ESP32-CAM

(IoT-Based Smart Surveillance System)

Submitted by: [Student Name]

Roll Number: [Roll No.]

Class / Semester: [Class, Semester]

Under the Guidance of: [Guide / Faculty Name]

Academic Year: [2025 - 2026]

[Department Name]
[College / Institute Name, City]

[Academic Year]

CERTIFICATE

This is to certify that the mini project report entitled **"Motion Detection and Face Identification using ESP32-CAM"** is a bonafide work carried out by **[Student Name]**, Roll No. **[Roll No.]**, in partial fulfilment of the requirements for the mini project component of **[Course / Program Name]** during the academic year **[2025-2026]**, under my/our supervision.

The project report has been approved as it satisfies the academic requirements prescribed for the said course.

_____

Signature of Guide

[Guide Name]

_____

Signature of Head of Department

[HOD Name]

_____

Signature of External Examiner

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide, **[Guide Name]**, for the continuous support, guidance, and valuable suggestions throughout the course of this mini project. I am also thankful to **[HOD Name]**, Head of the Department of [Department Name], and all the faculty members for providing the necessary resources and encouragement.

I would also like to thank my family and friends for their constant support and motivation during the completion of this project.

[Student Name]

TABLE OF CONTENTS

1.	Abstract	1
2.	Introduction	2
3.	Objectives	3
4.	Literature Survey / Existing Systems	3
5.	System Requirements	4
5.1	Hardware Components	4
5.2	Software Tools	5
6.	System Architecture and Block Diagram	6
7.	Working Methodology	7
8.	Circuit Diagram and Connections	9
9.	Implementation	10
10.	Results and Output	11
11.	Advantages	12
12.	Limitations	12
13.	Applications	13
14.	Future Scope	13
15.	Conclusion	14
16.	References	14

1. Abstract

This mini project presents the design and implementation of a low-cost, IoT-based motion detection and face identification system built around the ESP32-CAM module. The system is designed to monitor a specific area, detect motion using a Passive Infrared (PIR) sensor, capture an image through the onboard camera upon triggering, and transmit the captured frame over Wi-Fi to a server for face detection and identification. The server processes the received image using computer vision techniques to determine whether the detected face belongs to a known or an unknown individual, and the result is communicated back to notify the user.

The project demonstrates the integration of embedded systems, wireless communication, and computer vision to build a compact, affordable smart surveillance solution suitable for home security, restricted-area monitoring, and attendance-style identification applications, without relying on expensive dedicated CCTV or commercial smart-camera hardware.

2. Introduction

Security and surveillance have become essential parts of daily life, whether for protecting homes, offices, or restricted premises. Traditional CCTV-based surveillance systems, while effective, are often expensive, require dedicated recording hardware, and typically only provide passive video recording without any intelligent analysis of who is present in the frame.

With the growth of the Internet of Things (IoT) and the availability of low-cost, Wi-Fi-enabled microcontrollers with integrated cameras, it has become possible to build smart, connected surveillance systems at a fraction of the cost of commercial products. The ESP32-CAM, a compact module combining an ESP32 microcontroller with an OV2640 camera sensor and built-in Wi-Fi, is particularly well suited for such applications due to its low cost, small form factor, and ease of integration.

This project combines the ESP32-CAM's imaging and networking capabilities with a PIR motion sensor and a Python-based face recognition backend to create a system that not only detects movement but also identifies who triggered it — distinguishing between known, authorized individuals and unknown persons.

3. Objectives

1. To design a low-cost motion detection system using a PIR sensor interfaced with a microcontroller.
2. To capture images automatically using the ESP32-CAM upon detecting motion in the monitored area.
3. To transmit captured images wirelessly over Wi-Fi from the ESP32-CAM to a remote server.
4. To implement face detection and face recognition on the server side to identify whether a detected person is known or unknown.

5 To provide a real-time notification/response mechanism (visual indicator and/or alert) based on the identification result.

6 To build a working prototype that demonstrates the practical integration of embedded hardware, wireless communication, and computer vision.

4. Literature Survey / Existing Systems

Conventional surveillance systems primarily rely on CCTV cameras connected to a Digital Video Recorder (DVR) or Network Video Recorder (NVR). These systems continuously record footage, which must be manually reviewed to identify any person or event of interest. This approach consumes significant storage and does not provide any automated intelligence regarding who is present in the recorded footage.

More recent smart security products (e.g., smart video doorbells) offer motion alerts and cloud-based face recognition, but are typically closed, proprietary systems that require paid subscriptions and do not allow customization. In academic and hobbyist contexts, several projects have explored PIR-based motion detection and separate camera-based monitoring, but fewer combine both motion-triggered capture and automated face identification into a single low-cost, customizable pipeline — which is the gap this project aims to address using easily available open-source tools and inexpensive hardware.

5. System Requirements

5.1 Hardware Components

The following hardware components are used in this project:

S. No.	Component	Quantity	Purpose
1	ESP32-CAM (AI-Thinker, OV2640)	1	Captures images and streams data over Wi-Fi
2	FTDI / USB-to-TTL Programmer	1	Used to flash firmware onto the ESP32-CAM
3	PIR Motion Sensor (HC-SR501)	1	Detects motion in the monitored area
4	5V Power Supply / Adapter	1	Provides stable power to the ESP32-CAM
5	Jumper Wires	As required	Circuit connections
6	Breadboard	1	Prototyping the sensor circuit
7	LED / Buzzer (optional)	1-2	Local visual/audio alert on detection result

5.2 Software Tools

S. No.	Software / Library	Purpose
1	Arduino IDE	Writing and uploading firmware to the ESP32-CAM
2	ESP32 Board Package (Espressif)	Enables ESP32/ESP32-CAM support in Arduino IDE
3	esp32-camera Library	Interfacing with the OV2640 camera module
4	Python 3	Backend server for image processing
5	OpenCV	Face detection on the server
6	face_recognition (dlib-based)	Face identification / matching against known faces
7	Flask / FastAPI	Receiving images from ESP32-CAM over HTTP

6. System Architecture and Block Diagram

The system follows a simple, modular pipeline consisting of four stages: motion sensing, image capture, wireless transmission, and server-side face identification. The block diagram below illustrates the flow of data through the system.

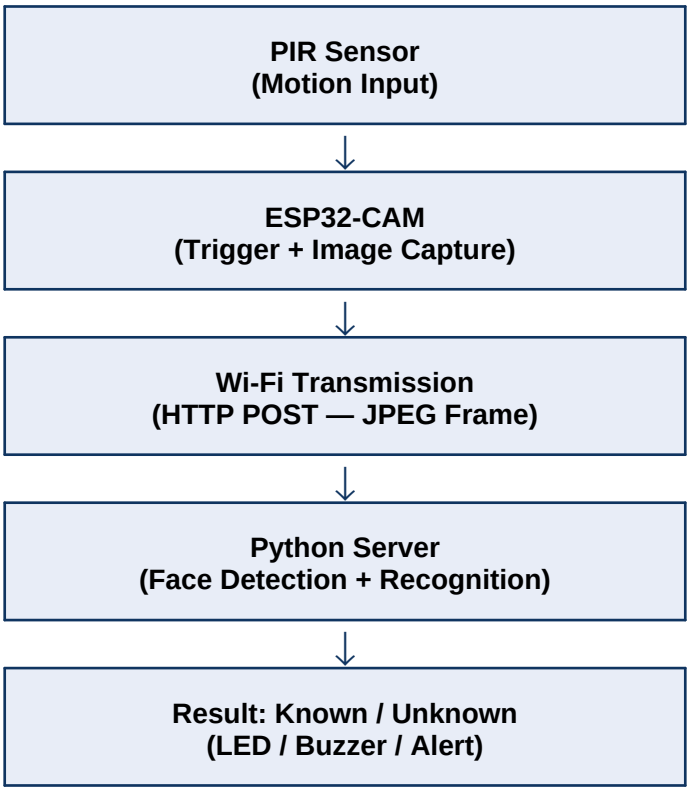

Fig. 6.1 — System block diagram showing data flow from motion detection to identification result

7. Working Methodology

The system operates in the following sequential stages:

7.1 Motion Detection

The PIR (Passive Infrared) sensor continuously monitors the surrounding area for infrared radiation changes caused by the movement of a warm body (such as a person). When motion is detected, the sensor's output pin goes HIGH, which is read by a GPIO pin, acting as the trigger signal for the capture process.

7.2 Image Capture

Upon receiving the motion trigger, the ESP32-CAM activates its OV2640 camera module and captures a JPEG-encoded frame. The captured frame is temporarily held in the module's frame buffer before being transmitted.

7.3 Wireless Transmission

The ESP32-CAM connects to the local Wi-Fi network (Station mode) and sends the captured JPEG image to a Python-based server using an HTTP POST request. This allows the computationally-light ESP32-CAM to offload the heavy face-recognition processing to a more capable server/computer.

7.4 Face Detection and Recognition

On receiving the image, the server uses OpenCV / the face_recognition library to first detect whether a face is present in the frame, and if so, extract its facial features and compare them against a pre-stored database of known face encodings. Based on the similarity score, the system classifies the detected face as either a **known** individual (with a matching identity) or an **unknown** individual.

7.5 Result and Notification

The identification result is sent back to notify the user — this may be implemented as a local indicator on the ESP32 side (LED/buzzer), a dashboard update, or an alert notification (e.g., Telegram/email) for unknown faces, depending on the deployment configuration.

8. Circuit Diagram and Connections

8.1 ESP32-CAM to FTDI (Programming Connections)

FTDI Pin	ESP32-CAM Pin	Purpose
5V	5V	Power supply during flashing
GND	GND	Common ground reference
TX	U0R (RX)	Data line (crossed)
RX	U0T (TX)	Data line (crossed)
—	GPIO 0 → GND	Enables flashing/bootloader mode (temporary jumper)

Note: GPIO 0 is grounded only during the firmware upload process to place the module into bootloader mode. It is disconnected during normal operation so the module boots and runs the uploaded program.

8.2 PIR Sensor to ESP32-CAM

PIR Sensor Pin	ESP32-CAM Pin	Purpose
VCC	5V	Power supply to PIR sensor
GND	GND	Common ground
OUT	GPIO (free pin, e.g. GPIO 13)	Motion trigger signal to microcontroller

Since most usable GPIOs on the ESP32-CAM are shared with the camera interface, only a limited number of pins (e.g., GPIO 12, 13, 14, 15, 16) are available for peripherals such as the PIR sensor, LED, or buzzer.

9. Implementation

The implementation was carried out in incremental stages, testing each subsystem individually before integrating them into the complete pipeline:

1. **Board bring-up and verification:** The ESP32-CAM and a standard ESP32 development board were tested individually to confirm correct chip identification, flash memory, and Wi-Fi connectivity before building further functionality on top.
2. **Camera and Wi-Fi web server:** The ESP32-CAM was flashed with firmware to initialize the OV2640 camera and host a simple web server, first tested to confirm it could serve a live video stream over the network.
3. **Motion sensor integration:** The PIR sensor was wired to a free GPIO pin, and firmware was written to detect the motion trigger signal and initiate image capture accordingly.

4. **Wireless image transmission:** The captured JPEG frame is sent from the ESP32-CAM to the server using an HTTP POST request over the local Wi-Fi network.
5. **Face detection and recognition backend:** A Python server was set up using OpenCV and the face_recognition library to receive incoming images, detect faces, and compare them against a database of known face encodings.
6. **Result feedback:** Based on the identification result, the system triggers a local indicator (LED/buzzer) and/or sends an alert notification for unrecognized individuals.

9.1 Firmware Overview

The ESP32-CAM firmware initializes the camera driver with the AI-Thinker pin configuration, connects to the configured Wi-Fi network, and either hosts a live MJPEG stream for testing purposes or captures single frames on a motion trigger for the full pipeline. Static IP configuration was used during testing to ensure the module remained reachable at a consistent network address.

9.2 Server-Side Processing Overview

The Python server exposes an endpoint to receive incoming JPEG frames. On receiving an image, it uses face detection to locate any face(s) present, generates a facial encoding, and compares it against a pre-built database of known encodings using a distance/similarity threshold. If the closest match falls within the threshold, the person is identified by name; otherwise, they are flagged as unknown.

10. Results and Output

The developed system was tested in a lab/home environment and successfully demonstrated the following outcomes:

- The ESP32-CAM successfully connected to the Wi-Fi network and hosted a live camera stream accessible over a browser.
- The PIR sensor reliably detected motion within its sensing range and triggered image capture.
- Captured images were successfully transmitted to the server over HTTP without significant loss or corruption.
- The face recognition backend correctly distinguished between known (registered) faces and unknown faces in test conditions.
- The end-to-end response time from motion trigger to identification result was within an acceptable range for a real-time alert system.

[Insert screenshots of the live camera stream, PIR-triggered capture, and face recognition output/console logs here.]

11. Advantages

- Low-cost solution compared to commercial smart-camera / CCTV systems.
- Compact and easy to deploy — requires only a small module, sensor, and power source.
- Wireless operation removes the need for long cabling runs.
- Modular design allows independent upgrading of the camera unit, sensor, or recognition backend.
- Can be customized and extended for specific use-cases (attendance, home security, restricted access).

12. Limitations

- The ESP32-CAM has limited processing power and memory, so all face recognition computation must be offloaded to an external server.
- PIR sensors can occasionally produce false triggers due to heat sources unrelated to human presence (e.g., sunlight changes, moving warm objects).
- System accuracy and response time depend on Wi-Fi network stability and signal strength.
- Face recognition accuracy can be affected by poor lighting conditions, camera angle, or low image resolution.
- Requires the ESP32-CAM and server to be on the same network (or reachable over the internet with additional configuration) for communication.

13. Applications

- Home security and entry-point monitoring.
- Restricted-area access monitoring in offices or labs.
- Automated attendance systems using face identification.
- Smart doorbell-style visitor identification.
- Low-cost surveillance for small businesses or personal use.

14. Future Scope

- Integrating cloud storage for logging detected events and captured images.
- Adding mobile app or Telegram/email-based real-time alert notifications for unknown faces.
- Improving recognition accuracy using deep-learning-based face embedding models.
- Adding multi-camera support for monitoring multiple entry points from a single server.
- Implementing on-device (edge) face detection to reduce network dependency, using optimized lightweight models where feasible.
- Battery and solar-power integration for fully wireless, outdoor deployment.

15. Conclusion

This mini project successfully demonstrates a functional, low-cost motion detection and face identification system using the ESP32-CAM. By combining a PIR-based motion trigger, Wi-Fi-based image transmission, and server-side face recognition, the system achieves intelligent, automated identification of individuals in a monitored area — going beyond simple motion-triggered recording offered by traditional systems. The project highlights the practicality of combining embedded hardware with computer vision techniques to build accessible smart-security solutions, and lays a solid foundation for further enhancement with cloud connectivity, mobile alerts, and improved recognition models.

16. References

1 Espressif Systems, "ESP32-CAM Development Board Documentation," Espressif Systems Official Documentation.

2 Espressif Systems, "esp32-camera Library," GitHub Repository, Espressif Systems.

3 OpenCV Documentation, "Open Source Computer Vision Library," opencv.org.

4 Adam Geitgey, "face_recognition Library," GitHub Repository.

5 Arduino Documentation, "Arduino IDE and ESP32 Board Support," arduino.cc.

6 [Add any additional textbooks, research papers, or online tutorials referred to during the project.]